포트폴리오 2026 / 개발자

실제 주문이 들어오는 자동 처리 시스템을 설계하고 멈추지 않게 직접 운영했습니다.

온라인 판매 주문 처리 서버를 설계해 첫 코드 5일 만에 판매를 시작했고, 판매 6일째 하루 1,378건을 처리했습니다. 결과가 불명확한 주문은 다시 보내지 않고 보류한 뒤 거래 이력과 맞춰 확정하게 했고, 돈이 나가는 일과 AI 에이전트에게 맡길 일을 나눠 한도를 정했습니다.

현장실습에서는 고칠 수 없는 출입 시스템의 기록만 읽어 근무 상태를 판정하는 연동을 회사 서버에 배포했고, 3인 팀장으로 공공기관 복지 챗봇을 RAG로 다시 만들었습니다.



숫자로 보면

1,378건
하루 최대 주문 · 판매 6일째

5일
첫 코드에서 판매 시작까지

팀장
3인 · 공공기관 협력 RAG 챗봇

1,809건
복지사업 · 매일 새벽 자동 반영

수상

2025 컴퓨터인공지능학부 작품경진대회 은상
2025 캡스톤디자인 우수상
2025 전북특별자치도사회서비스원 성과보고회 표창

학력 · 자격

전북대 컴퓨터공학 · 전공 3.83 / 전체 3.71 (4.5)
정보처리기사 · SQLD · ADsP

배지혁

전북대학교 컴퓨터공학부 / qowlgur121@gmail.com / github.com/qowlgur121 / qowlgur121.github.io

소개

학력 · 현장실습 · 기술 · 푼 문제

학력

전북대학교	컴퓨터공학부 공학사 · 2022.03 ~ 2026.02
성적	전체 3.71/4.5 · 전공 3.83/4.5 3학년 2학기 4.50 (학과 1등)

현장실습

(주)텔로스	백엔드 현장실습 · 2025.07 출입 기록으로 근무 상태를 판정하는 배치 · 회사 서버 배포

수상

2025.12	작품경진대회 은상 · 전북대 컴퓨터인공지능학부
2025.12	캡스톤디자인 우수상 · 전북대학교
2025.11	성과보고회 표창 · 전북특별자치도사회서비스원

자격증 · 활동

자격증	정보처리기사 · SQLD · ADsP
JAVICE	SW 창업동아리 · 2023.09 ~ 2025.12
ALPS	알고리즘 동아리 스터디 멘토 · 2022.03 ~ 2023.07

기술

Java	현장실습 · OpenContext · JeonjuEats 백엔드
Spring Boot	현장실습 출입 판정 배치 · OpenContext 검색 API
Python	주문 자동화 시스템 · 운영 스크립트
FastAPI	복지 AI 챗봇 백엔드 · SSE 스트리밍
PostgreSQL	pgvector 벡터 검색 · OpenContext 메타데이터
Docker	Docker Compose · Nginx로 서버 배포

이 포트폴리오에서 푼 문제

결과를 모르는 외부 요청	재시도 대신 보류 → 거래 이력 대조 → 확정 · 주문 자동화
같은 코드를 동시에 잡는 두 처리	UNIQUE · 조건부 UPDATE · 한 트랜잭션 · 주문 자동화
AI 에이전트에 맡길 범위	처리는 코드 · 감시는 에이전트 · 돈과 한도는 사람 · 주문 자동화
새 사업마다 재학습하던 챗봇	매일 갱신되는 RAG · pgvector · 복지 챗봇

01

주문 자동화 시스템과 온라인 스토어 운영

대표 프로젝트 · 개요와 구조

개인 프로젝트 · 기획부터 운영까지 직접

2026.09 ~ 2026.10

온라인 판매를 열기 전에 주문 처리 서버부터 설계했습니다. 결제 확인부터 발주·전달·정산 대조까지 이어지게 해 판매 6일째 하루 1,378건을 처리했습니다.

처음 정한 조건	서버 비용은 최소로 · 보내기 전에 상품 코드가 유효한지 확인 · 운영 부담은 제가 감당할 수 있게
Stack	Python · SQLite WAL · unittest 회귀 테스트
Infra	Oracle Cloud ARM VM · Ubuntu · systemd 타이머 · 전용 계정과 ProtectSystem=strict
연동	네이버 커머스 API · 외부 주문 API · 문자 발송 · Telegram 알림
역할 분리	주문 처리는 정해진 코드로만 · AI 점검 에이전트(15분)는 상태를 읽고 허용된 조치만

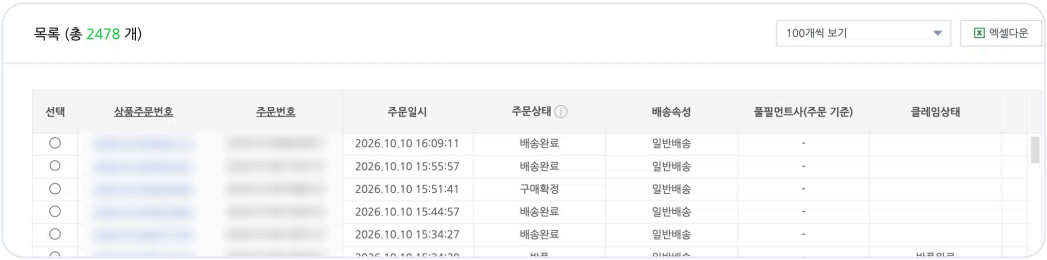
1,378건
하루 최대 주문 (10/07) · 9일 누적 2,469건

5일
첫 코드 → 판매 시작

20초
주문 조회 주기 · 백업 매일



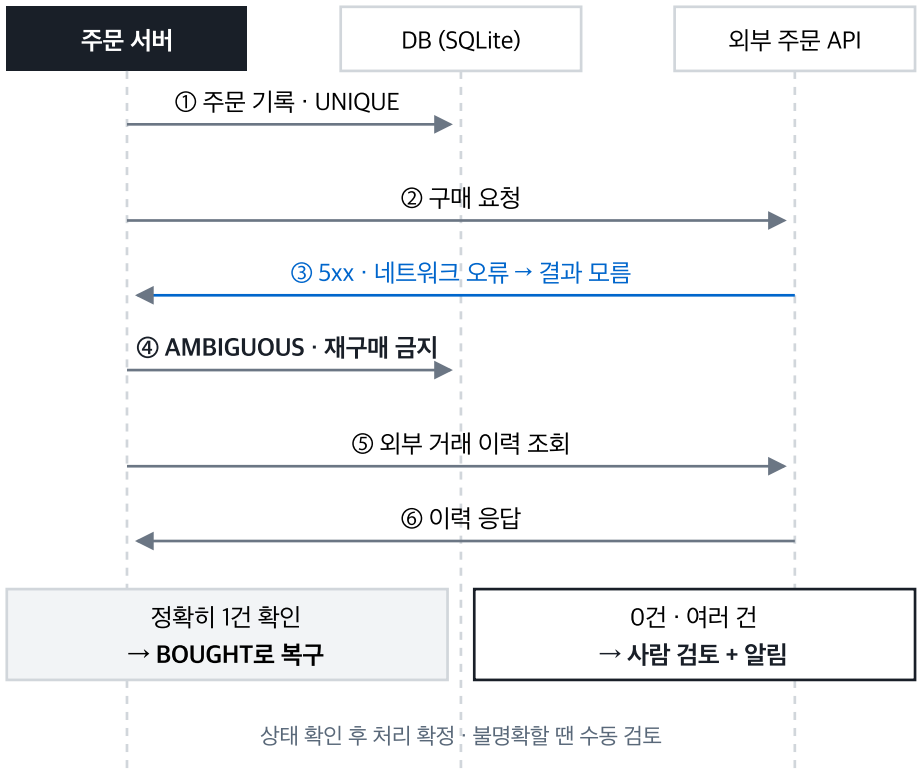
제가 나눈 시스템 구조. 회색 상자는 AI 에이전트 · 테두리 상자는 사람 판단



판매자센터 주문 목록 (주문번호는 가림)

응답이 끊긴 주문은 다시 보내지 않고, 보류한 뒤 거래 이력으로 확정하게 설계했습니다

문제 해결 1



구매 결과를 모를 때의 처리 순서

문제

판매 초기에 "같은 사람이 한 번에 여러 개를 사거나 연달아 사도 문제없어야 한다"를 조건으로 정했습니다. 그런데 외부 주문 API의 응답이 끊기면 처리됐는지 알 수 없었고, 그냥 재시도하면 같은 주문이 두 번 처리될 수 있었습니다.

설계 판단

자동 재시도	동일 요청이 중복 실행될 수 있음
즉시 실패	실제 처리 여부와 내부 상태가 어긋날 수 있음
선택한 방식	**상태 보류 → 거래 이력 확인 → 결과 확정**

결과가 불명확하면 **AMBIGUOUS**로 보류하고, 외부 거래 이력에서 일치하는 내역을 찾아 복구했습니다. 애매한 건은 사람 확인으로 넘겼습니다.

주문번호 **UNIQUE** 제약과 조건부 UPDATE로 동일 주문의 상태 전이를 보호했습니다.

결과

불명확한 주문을 재실행하지 않고 **대조 후 확정**하는 순서로 판매 6일째 하루 1,378건을 처리했습니다. 발송 직전에도 같은 1회용 상품 코드가 다른 주문에 나간 적이 있는지 한 번 더 확인하고, 겹치면 보내지 않고 멈추게 했습니다. 잘못 보내는 것보다 멈추는 편이 낫다고 판단했습니다.

서버 1대(Oracle Cloud ARM 1 OCPU · 6GB) · SQLite WAL · 20초 주기 조회
핵심 로직 예제 · github.com/qowlgur121/order-fulfillment-guard

두 처리가 같은 코드를 동시에 잡아도 한쪽만 성공하게, 조건부 UPDATE와 트랜잭션으로 막았습니다

문제 해결 2

① 같은 주문이 두 번 들어와도 한 건만

```
UNIQUE INDEX orders(channel, channel_order_id)
```

20초마다 조회해 같은 주문을 또 읽어도 새로 생기지 않음

② 같은 상품 코드는 한 번만 등록

```
artifacts.fingerprint UNIQUE
```

공급처가 같은 코드를 두 번 돌려줘도 두 번째는 거부

③ 한 코드는 한 주문에만 (조건부 UPDATE)

```
UPDATE artifacts SET state='RESERVED',
reserved_order_id=?
WHERE id=? AND state='VALIDATED'
AND reserved_order_id IS NULL;
```

바뀐 행이 1개가 아니면 예외 → 트랜잭션 전체 되돌림

상태 변화는 events 표에 모두 기록

어느 주문에 어떤 코드가 언제 묶였는지 추적 가능

중복을 DB가 마지막에 막는 구조

문제

주문 조회는 20초마다, 점검은 15분마다 따로 돌았습니다. 같은 주문을 두 번 읽거나, 두 처리가 같은 1회용 코드를 동시에 잡으면 **한 코드가 두 고객에게 가거나 같은 주문에 코드가 두 번** 나갈 수 있었습니다.

설계 판단

코드에서 먼저 조회 후 저장	조회와 저장 사이에 다른 처리가 끼어들 수 있음
별도 잠금 서버 · 큐	서버 1대 규모에 운영할 것만 늘어남
선택한 방식	**DB 제약 + 조건부 UPDATE로 DB가 마지막에 막게**

SQLite를 WAL 모드로 두고, 코드 할당과 주문 갱신을 **한 트랜잭션**으로 묶었습니다. 발송 기록 저장이 실패하면 코드 할당까지 함께 되돌립니다.

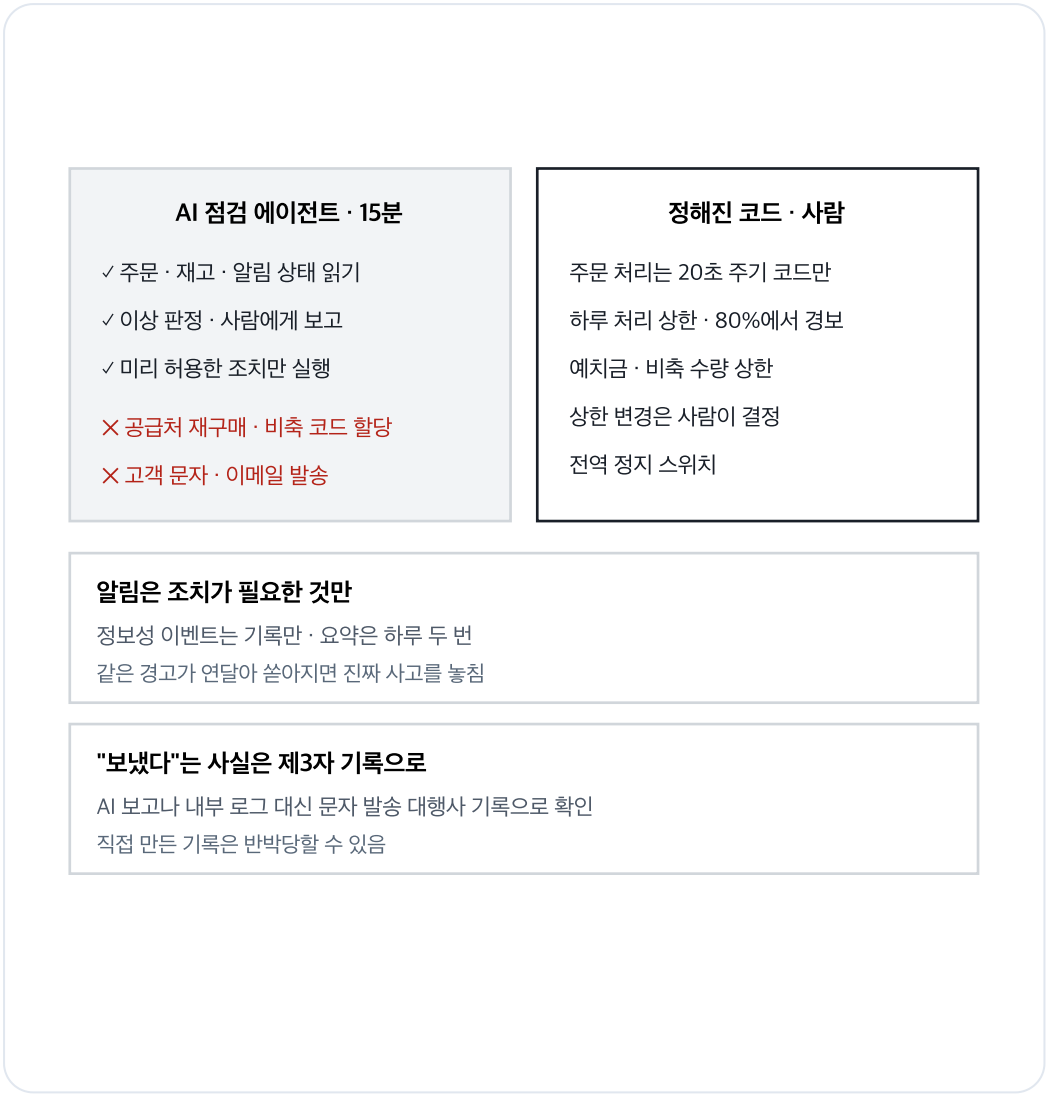
결과

두 처리가 같은 코드를 동시에 잡아도 **한쪽만 성공**하고 나머지는 예외로 멈춥니다. 중복 코드 거부 · 원자 할당 · 실패 시 되돌림을 각각 테스트로 고정했습니다.

Python sqlite3 · WAL · UNIQUE 제약 · 조건부 UPDATE · changes() 검사

AI 점검 에이전트를 운영에 붙이며, 할 수 있는 일과 못 하는 일을 먼저 정했습니다

문제 해결 3



AI 에이전트 · 코드 · 사람의 역할 나누기

문제

혼자 24시간 지켜볼 수 없어 AI 점검 에이전트를 붙였습니다. 하지만 1회용 코드와 선결제가 걸린 일이라, 에이전트가 잘못 판단하면 **되돌릴 수 없는 손해**가 납니다.

설계 판단

에이전트가 처리까지	같은 상황에서도 매번 다르게 판단할 수 있음
사람이 모든 주문 확인	하루 1,000건 넘게 오면 감당 불가
선택한 방식	**처리는 코드, 감시는 에이전트, 돈 · 한도는 사람**

에이전트가 해서는 안 되는 일(재구매 · 코드 할당 · 고객 발송)을 목록으로 막고, 규칙으로 처리할 수 있는 일에는 AI를 부르지 않았습니다.

결과

에이전트는 상태를 읽고 보고하는 역할에 두고, **돈이 나가는 조치는 모두 정해진 코드 규칙과 제 결정으로만** 실행되게 한 채 판매 6일째 하루 1,378건을 처리했습니다. 처리 상한은 정상 판매가 확인될 때만 늘어나게 했습니다.

02

공공 복지 AI 챗봇 고도화 · RAG

대표 프로젝트 · 개요와 구조

산학 프로젝트 · 3인 팀 팀장

2025.09.22 ~ 2025.12.05

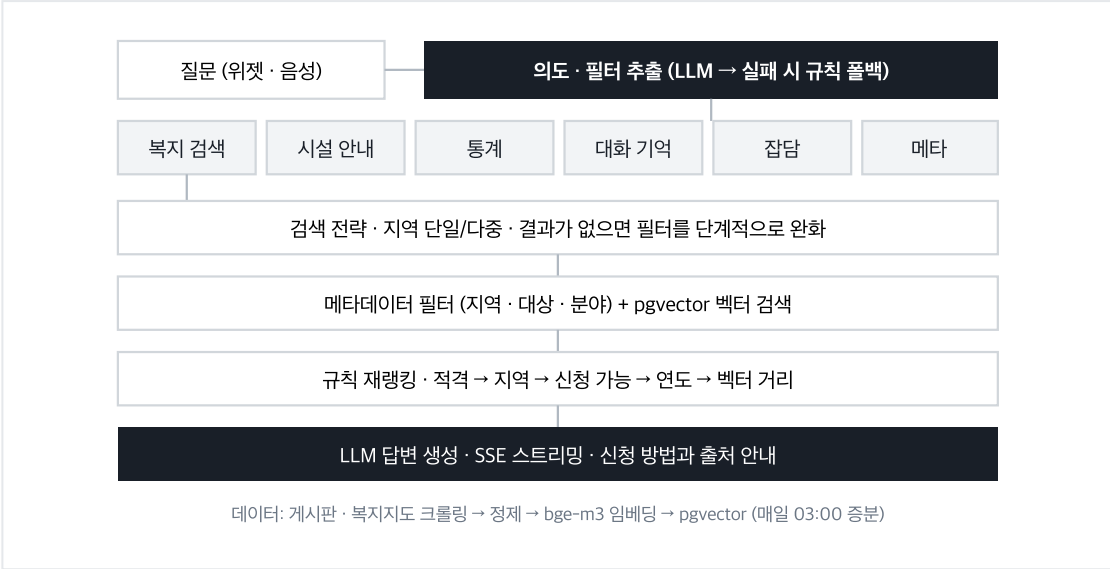
표현이 조금만 달라도 답을 못 찾던 기관 챗봇을 의도 판단과 RAG 검색으로 다시 만들었습니다. 팀장으로 기관과 요구사항을 맞추고 개발을 맡았습니다.

협력	전북특별자치도사회서비스원
처음 정한 조건	기관 원본 시스템은 고칠 수 없음 · 운영 담당자는 개발자가 아님 · 회원 개인정보는 받지 않음
Backend	FastAPI · SQLAlchemy · APScheduler · SSE · Docker Compose · 위젯 Custom Element
AI · 검색	LangChain 0.3 · bge-m3 임베딩 · PostgreSQL + pgvector · LLM 클라우드/로컬 선택
역할	**내 담당** 요구사항 협의 · 설계 · 백엔드 · 검색 · 관리자 · 위젯 · 발표 **팀원** 자료 조사 · 발표 자료 · 문서

1,809건
복지사업 · 매일 03시 갱신

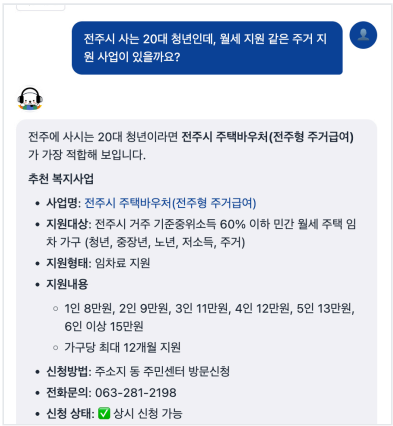
18,318곳
복지시설 위치 데이터

우수상
캡스톤 우수상 · 기관 표창



데이터: 게시판 · 복지지도 크롤링 → 정제 → bge-m3 임베딩 → pgvector (매일 03:00 증분)

질문 하나가 답이 되기까지의 흐름





조건을 말하면 받을 수 있는 사업과 신청 방법 안내 / 증상을 말하면 가까운 의료기관을 거리순 안내

새 사업마다 재학습이 필요했던 챗봇을 매일 갱신되는 RAG 구조로 바꿨습니다

문제 해결 4



기존 방식과 새 방식 비교

문제

기존 챗봇은 파인튜닝 기반이라 학습한 Q&A 안에서만 답했고, 새 사업을 넣으려면 다시 학습해야 해서 신청 기간이 지난 사업을 안내하기도 했습니다.

해결

재학습 주기화 학습 비용이 들고 그 사이 정보는 계속 낡음

채택 **모델은 그대로 두고 매일 갱신되는 데이터를 찾아 답하는 RAG**

데이터 연동 API를 받을 수 없어 처음엔 기관 엑셀을 기준으로 삼으려 했지만, 공개 게시판을 직접 수집할 수 있음을 확인하고 **엑셀 의존을 걷어냈습니다.** 공공기관 사이트라 Spring으로 가면 전자정부 프레임워크까지 맞춰야 할 수 있어서 수집·검색 서버는 Python 하나로 묶었습니다.

바뀐 사업만 매일 수집해 500자 이하로 나누고, 전용 벡터 DB 대신 PostgreSQL에 pgvector를 붙여 **기관이 관리할 시스템을 하나로** 줄였습니다.

결과

재학습 없이 복지사업 1,809건이 매일 새벽 반영되고, 찾은 원문과 공식 게시글 링크로 답합니다. 키워드 검색으로 0건이던 "치매" 질문에서 관련 사업 20건을 찾았습니다.

OpenContext · 사내 문서를 밖으로 보내지 않는 AI 검색

개요와 구조

3인 팀 · 오픈소스 개발자대회

2025.08 ~ 2025.12 · github.com/OpenContextAI/open-context

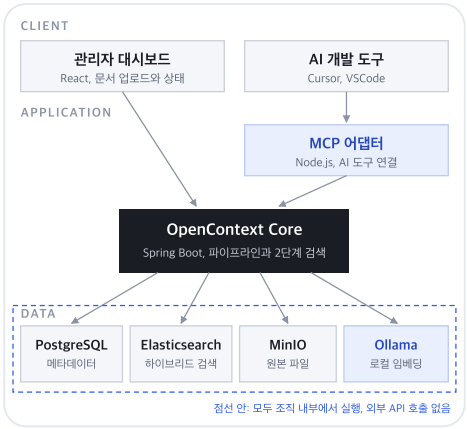
외부 AI 문서 서비스는 통제권이 밖에 있었습니다. 그래서 회사가 자기 서버에 설치해 쓰는 오픈소스를 제안해 팀원들과 만들었습니다.

Backend	Java 21 · Spring Boot 3.3 · QueryDSL · Flyway · LangChain4j
검색	Elasticsearch 8.11 · Ollama bge-m3 로컬 임베딩 · MinIO · PostgreSQL 16
연동	MCP (TypeScript SDK) · Cursor · VS Code
Admin	React 19 · Vite · TanStack Query · Zustand · Tailwind
역할	**내 담당** 제안 · 하이브리드 검색 엔진 · 2단계 검색 API · DB 모델링과 테스트 · 관리자 화면 · Docker 배포 · 보안 패치 **팀원** 문서 수집 파이프라인 · MCP 어댑터 · 공통 에러 처리

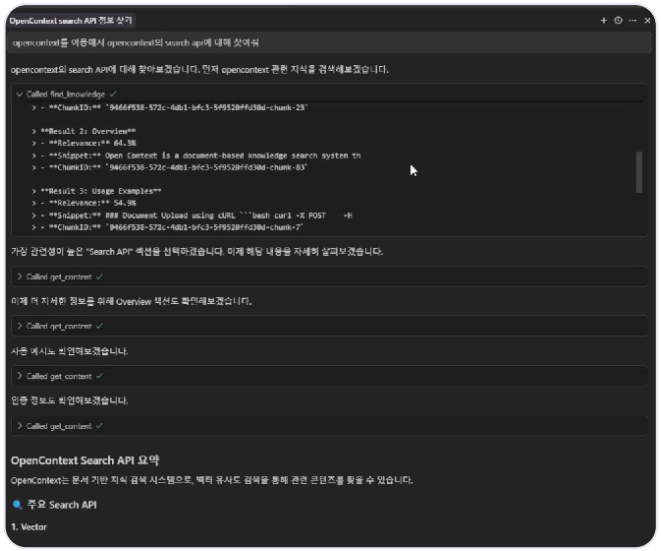
은상
학부 작품경진대회

로컬 처리
문서 처리 · 임베딩을 조직 안에서

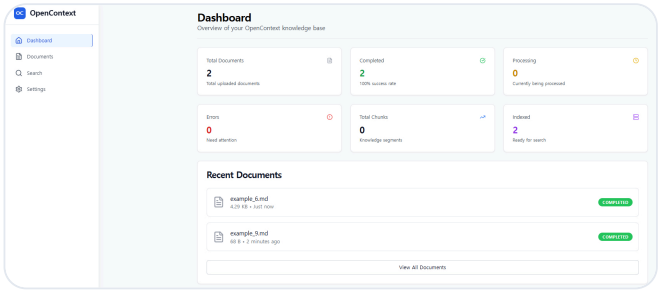
2단계
고르고 원문만 가져오기



전체 구조. 점선 안은 모두 조직 내부에서 실행



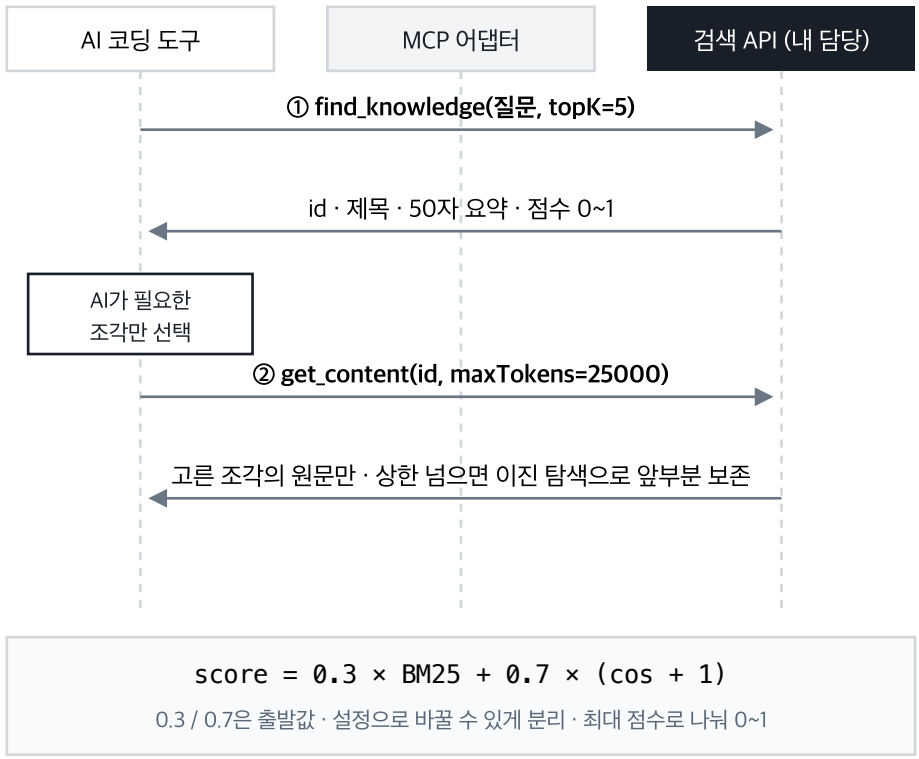
AI 도구가 find_knowledge로 고르고 get_content로 원문만 가져옴



제가 만든 관리자 화면 · 문서 처리 상태 확인

AI에 필요한 문서만 전달하는 2단계 검색 API를 설계했습니다

문제 해결 5



2단계 검색 호출 순서와 점수 계산

문제

검색한 원문을 한 번에 다 넘기면 컨텍스트가 커지고 AI가 쓸 자료를 고르기 어려웠습니다.

해결

find_knowledge에서 제목·요약·관련도 목록을 먼저 반환하고, AI가 선택한 문서만 **get_content**로 원문을 가져가도록 나눴습니다.

검색은 BM25 키워드 점수와 벡터 유사도를 합치고, 순위와 함께 관련도 점수를 돌려줬습니다.

> **핵심 선택** · 검색 결과 탐색과 원문 읽기를 분리해 필요한 컨텍스트만 사용

결과

AI 도구가 **먼저 고르고 필요한 원문만 읽는 검색 경로**를 만들었습니다. 문서 처리와 임베딩은 조직 안에서만 돌게 했고, 학부 작품경진대회 은상을 받았습니다.

제 담당 · 검색·조회 API · DB · Docker 배포 · 관리자 화면 (팀 커밋 162개 중 64개)